Linux网络--网络层路由表、逐跳转发与ICMP诊断

- Linux网络--网络层路由表、逐跳转发与ICMP诊断
- 1. 路由的本质
  - 1.1 选择的是下一跳，不是整条路
  - 1.2 目的 IP 与下一跳 IP
- 2. 主机路由表
  - 2.1 主要字段
  - 2.2 路由表示例
  - 2.3 两个转发例题
    - 2.3.1 访问 192.168.56.3
    - 2.3.2 访问 202.10.1.2
  - 2.4 最长前缀匹配
  - 2.5 Linux 路由观测
  - 2.6 策略路由
  - 2.7 路由缓存与邻居缓存
  - 2.8 路由和源地址选择
- 3. 公网分层路由模型
  - 3.1 三级结构示例
  - 3.2 层级路由怎样减少查找范围
  - 3.3 典型转发过程
- 4. 一跳里发生了什么
  - 4.1 三层与二层的变化
  - 4.2 为什么下一跳 IP 还不够
- 5. 路由表从哪里来
  - 5.1 静态路由
  - 5.2 动态路由
    - 5.2.1 自治系统
  - 5.3 默认路由
    - 5.3.1 默认路由不是万能路由
    - 5.3.2 多条等价路由与优先级
    - 5.3.3 route 与 ip route
- 6. 路由为什么有时会失败
  - 6.1 路由表命中后的下一阶段
- 7. ICMP：路由失败时谁能报告原因
  - 7.1 ICMP 的定位
  - 7.2 ICMP 并不是可靠传输

    - 7.3 ping
    - 7.4 ping 的边界
    - 7.5 traceroute
    - 7.6 ICMP 与路径 MTU
    - 7.7 一次合理的诊断顺序
- 8. 本篇小结

Linux网络--网络层路由表、逐跳转发与ICMP诊断

系列导航

上一篇：Linux网络--网络层私有IP、NAT与公网访问
本篇：路由表、逐跳转发与 ICMP 诊断
下一篇：Linux网络--数据链路层以太网帧与交换机

经过 NAT 后，报文已经获得了可以在公网使用的源地址。接下来的问题是：源主机和每一台路由器怎样决定下一步把报文交给谁？

答案不是“全球有一个总服务器告诉所有报文怎么走”，而是每个节点维护自己的路由表。路由器根据目的 IP 做最长前缀匹配，确定下一跳和发送接口；逐台路由器重复这个过程，最终到达目的网络。

1. 路由的本质

1.1 选择的是下一跳，不是整条路

源主机通常不需要提前计算从起点到终点的完整路径。它只需要把报文交给一个合适的下一跳；下一跳再用自己的路由表继续转发。

```
源主机 -> 路由器 A -> 路由器 B -> 路由器 C -> 目的主机
```

每一跳只负责当前决策：

1. 读取目的 IP；
2. 查询路由表；
3. 选择最长前缀匹配条目；

4. 获取下一跳；
5. 通过对应接口发送。

这种模式称为逐跳转发。

1.2 目的 IP 与下一跳 IP

地址	含义
目的 IP	最终目标主机或服务的地址
下一跳 IP	当前节点把报文交给的下一台三层设备

两者通常不同。目的 IP 在整条路径上保持不变，下一跳 IP 会随着当前路由器变化。发生 NAT 时，源或目的地址可能会有例外，但普通路由转发不修改最终目的 IP。

2. 主机路由表

2.1 主要字段

Linux/Unix 风格的路由表通常包含：

字段	含义
Destination	目的网络
Gateway	下一跳地址
Genmask	子网掩码
Flags	条目属性
Iface	发送接口
Metric	路径优先级

常见标志：

- `U`：条目有效；
- `G`：需要通过网关；
- 没有 `G`：目标网络与接口直接相连。

2.2 路由表示例

Destination	Gateway	Genmask	Flags	Iface
`192.168.10.0`	`*`	`255.255.255.0`	U	eth0
`192.168.56.0`	`*`	`255.255.255.0`	U	eth1
`127.0.0.0`	`*`	`255.0.0.0`	U	lo
`default`	`192.168.10.1`	`0.0.0.0`	UG	eth0

第一、二行是直连网络。第三行表示回环地址。最后一行是默认路由，通常指向本机所在网络的网关。

2.3 两个转发例题

2.3.1 访问 `192.168.56.3`

依次用条目的掩码与目的地址做按位与：

```
192.168.56.3 AND 255.255.255.0
= 192.168.56.0
```

匹配第二行，因此从 `eth1` 直接发出，不需要经过路由器网关。

2.3.2 访问 `202.10.1.2`

该地址不匹配前面任何直连网络，只能命中默认路由：

```
default -> 192.168.10.1 via eth0
```

报文先发给 `192.168.10.1`；之后由下一台路由器自己的表继续查找。

2.4 最长前缀匹配

多条路由都可能包含某个目的地址时，路由器选择网络前缀最长、范围最具体的那条。

例如：

```
10.0.0.0/8        -> 下一跳 A
10.1.0.0/16       -> 下一跳 B
10.1.2.0/24       -> 下一跳 C
```

目的地址 `10.1.2.9` 同时命中三条，但 `/24` 最具体，因此选择下一跳 C。

2.5 Linux 路由观测

现代 Linux 使用：

```
ip route
ip route get 6.0.100.200
```

`ip route get` 会直接展示内核选择的路由、出口设备和源地址：

```
6.0.100.200 via 192.168.1.1 dev eth0 src 192.168.1.20
```

这比只阅读完整路由表更容易回答：

- 命中哪条路由；
- 从哪个接口出去；
- 下一跳是谁；
- 本机使用哪个源地址；
- 是否被策略路由改变。

传统 `route -n` 仍可阅读，但新系统优先使用 `ip route`。

2.6 策略路由

普通路由主要看目的地址，但 Linux 还可以根据源地址、接口、防火墙标记等规则先选择不同的路由表。

```
ip rule
ip route show table main
```

```
ip route show table local
```

多网卡、VPN、容器和双出口环境经常使用策略路由。同一个目的地址从不同源或不同接口访问时，可能走出不同路径。

如果“本机 curl 正常，但容器或 VPN 内失败”，不能只检查默认路由，还要检查路由规则和所属网络命名空间。

2.7 路由缓存与邻居缓存

Linux 不再使用传统的长期 IPv4 路由结果缓存作为主要机制，但邻居解析仍然影响发送：

```
ip neigh
```

当路由选中下一跳后，邻居表需要提供 MAC。路由正确但邻居解析失败时，报文仍然无法封装。

2.8 路由和源地址选择

路由结果还可能决定源地址。主机访问远端服务时，可能根据出口接口自动选择源 IP；NAT 发生前，路由表看到的源地址与公网服务端最终看到的地址可能不同。

完整排查应同时查看：

```
目的路由
源地址选择
NAT 规则
连接状态
ARP/邻居状态
```

3. 公网分层路由模型

3.1 三级结构示例

用国家、省、市近似解释互联网的多级地址规划：

```
中国      5.0.0.0/8
陕西      5.1.0.0/16
西安      5.1.16.0/20
```

类似的示例还有：

- 俄罗斯 `1.0.0.0/8`
- 美国 `2.0.0.0/8`
- 英国 `3.0.0.0/8`
- 法国 `4.0.0.0/8`
- 意大利 `6.0.0.0/8`

国内省份分别使用：

```
5.1.0.0/16  陕西
5.2.0.0/16  河南
5.3.0.0/16  浙江
5.4.0.0/16  吉林
5.5.0.0/16  河北
```

省份内部再使用额外主机位划分城市：

```
5.1.16.0/20  西安
5.1.32.0/20  延安
5.1.64.0/20  咸阳
```

`/20` 对应的正确掩码是 `255.255.240.0`。

3.2 层级路由怎样减少查找范围

报文到达中国国际边界时，先根据 `5.0.0.0/8` 确定目的国家；到达国内骨干后，再根据 `/16` 前缀确定省份；到达省级网络后，再根据 `/20` 前缀确定城市。

每一层只暴露必要的汇总信息，不必把所有主机地址都堆在一个表里。这就是路由聚合的意义。

3.3 典型转发过程

假设俄罗斯主机访问 `5.1.16.100`：

1. 俄罗斯路由器发现目的地址不属于本地网络；
2. 根据国际路由表找到 `5.0.0.0/8`，下一跳是中国国际边界；
3. 中国边界继续查找 `5.1.0.0/16`，交给陕西省级网络；
4. 陕西路由器查找 `5.1.16.0/20`，交给西安网络；
5. 西安入口进入内网路由或城域网，最终到达目的主机。

真实互联网由自治系统、BGP 前缀、运营商策略和 IXP 构成。国家、省、市图是帮助理解分层的简化模型，不应理解为每台路由器都保存一张真实国家地图。

4. 一跳里发生了什么

4.1 三层与二层的变化

一台路由器收到帧时：

1. 链路层检查目的 MAC；
2. 拆掉当前帧的链路层首尾；
3. 网络层读取目的 IP；
4. 路由表决定下一跳；
5. TTL 减 1；
6. 通过下一跳 IP 解析对应 MAC；
7. 重新封装新链路的帧。

跨越多台路由器时：

```
目的 IP：通常保持不变
TTL：逐跳减少
源/目的 MAC：每跳重写
链路层帧：每跳重新封装
```

4.2 为什么下一跳 IP 还不够

路由表只能给出下一跳 IP，链路层却必须填写 MAC。于是每个广播域内还需要 ARP 将下一跳 IP 转换为 MAC。

这个关系会在数据链路层和 ARP 篇中详细展开。

5. 路由表从哪里来

5.1 静态路由

网络管理员手工配置，适合拓扑稳定、路径明确的网络。

5.2 动态路由

路由器通过协议交换可达信息并自动计算路径，常见算法包括：

- 距离向量；
- 链路状态；
- Dijkstra 最短路径。

互联网中不同自治系统之间主要使用 BGP。BGP 传递的是网络可达性和路径策略，并不等同于简单“地理距离最短”。

5.2.1 自治系统

一个由单一机构管理、采用统一路由策略的网络集合可以看作自治系统（AS）。每个 AS 使用 AS Number 标识。

BGP 在 AS 之间传播前缀可达性，并携带 AS_PATH、下一跳、本地优先级和社区等属性。路径选择不是简单的“经过路由器最少”，还会受商业关系、策略和冗余设计影响。

课堂中的“国家路由器、省级路由器”是对层级和地址聚合的形象化，不能等同于真实 BGP 拓扑。

5.3 默认路由

默认路由是前缀长度 0 的兜底条目：

```
0.0.0.0/0 -> 默认网关
```

只有所有更具体路由都不匹配时，才使用默认路由。出口路由器、家庭网关和大多数客户端都会配置默认路由。

5.3.1 默认路由不是万能路由

默认路由只是把未知目的交给一个默认网关。如果该网关没有继续可达的路径，报文最终仍会被丢弃。

默认路由也不能覆盖：

- 更具体的本地直连路由；
- 更具体的 VPN 路由；
- 防火墙策略；
- 目标主机离线；
- 回程路由缺失。

5.3.2 多条等价路由与优先级

同一目的前缀可能存在多条路径：

```
10.20.0.0/16 via 192.168.1.2 metric 10
10.20.0.0/16 via 192.168.2.2 metric 20
```

路由实现通常先比较前缀长度，再比较管理距离或 metric。不同来源的路由还需要比较优先级，例如直连、静态、内部动态协议和外部 BGP 路由可能有不同默认优先级。

Linux 可以在一个路由表中安装多条权值相同的下一跳，形成多路径转发：

```
ip route add 10.20.0.0/16 \
    nexthop via 192.168.1.2 dev eth0 weight 1 \
    nexthop via 192.168.2.2 dev eth1 weight 1
```

多路径并不等同于链路聚合。每条流可能仍只选择一个下一跳，只有多个流才可能被分散到不同路径。

5.3.3 route 与 ip route

传统：

```
route -n
```

现代：

```
ip route show
```

两者表达同一类信息，但 `iproute2` 支持多路由表、策略路由、多路径和更丰富的属性，因此 Linux 网络排错应优先使用 `ip` 命令。

6. 路由为什么有时会失败

路由表存在条目，不代表报文一定能到终点。常见失败原因包括：

- 目的前缀根本不存在；
- 只配置了默认路由，但默认网关不可达；
- 去程可达、回程缺少路由；
- 路由环路使 TTL 到期；
- ACL 或防火墙在某一跳丢弃；
- 下一跳 IP 可达，但 ARP 解析失败；
- MTU 不一致但 ICMP 被过滤；
- NAT 映射状态缺失或过期。

因此“路由正确”只是一个必要条件，而不是端到端可达的充分条件。

6.1 路由表命中后的下一阶段

路由器找到下一跳 IP 之后，还要完成：

```
下一跳 IP -> 下一跳 MAC
```

如果下一跳在同一广播域，路由器使用 ARP；如果下一跳是自己直连接口，也要解析对应链路地址。只有拿到 MAC 后，才能构造新的链路层帧。

这解释了为什么路由表和 ARP 是连续的两步：前者处理三层逻辑路径，后者完成当前链路的实际交付。

7. ICMP：路由失败时谁能报告原因

7.1 ICMP 的定位

ICMP（Internet Control Message Protocol）直接承载在 IPv4 数据报中，IPv4 首部协议号是 1。它不是 TCP/UDP，不使用端口。

```
IPv4 数据报
    └── ICMP 消息
```

ICMP 主要分为差错通知和诊断查询：

类型示例	含义
Echo Request / Echo Reply	请求和回应连通性探测
Destination Unreachable	网络、主机、协议或端口不可达
Time Exceeded	TTL 到期或重组超时
Redirect	通知存在更优网关
Fragmentation Needed	报文超过路径 MTU，需要降低大小

有些早期类型已经废弃，例如 Source Quench 和地址掩码请求/应答。IPv6 使用 ICMPv6，协议号 58，并承担邻居发现、路由器发现等重要功能。

7.2 ICMP 并不是可靠传输

ICMP 可以报告错误，但：

- 路由器不保证一定发送 ICMP；
- 防火墙可能过滤 ICMP；
- ICMP 本身也可能丢失；
- 收到 ICMP 错误不等于原始报文一定没有被部分处理；
- ICMP 不负责重传。

它提供的是控制反馈，而不是可靠传输。

7.3 ping

`ping` 发送 ICMP Echo Request：

```
主机 A -> Echo Request -> 主机 B
主机 A <- Echo Reply   <- 主机 B
```

Linux：

```
ping -c 4 example.com
```

Windows：

```
ping -n 4 example.com
```

常见输出信息：

- 收到的字节数；
- 往返时间；
- TTL；
- 丢包率；
- 最小、最大和平均 RTT。

`TTL` 是收到回复时剩余的 TTL，不一定是“经过了多少路由器”。初始 TTL 可能因操作系统和协议栈实现不同而变化。

7.4 ping 的边界

`ping` 成功只能说明：

- 至少有一部分 ICMP 探测到达目标；
- 目标或中间设备愿意返回回复；
- 当前路径具备基本可达性。

它不能证明：

- HTTP、SSH 或其他 TCP 服务正在监听；
- 应用层协议配置正确；

- 登录凭据有效；
- 业务返回 200；
- 大报文路径没有问题；
- 去程与业务流量经过完全相同的路径。

`ping` 失败也不一定代表目标离线，可能只是 ICMP 被过滤。

7.5 traceroute

`traceroute` 利用 TTL：

1. 发出 TTL=1 的探测报文；
2. 第一台路由器把 TTL 减为 0，返回 ICMP Time Exceeded；
3. 再发 TTL=2，获得第二台路由器响应；
4. 依次增加 TTL，直到目标回复。

不同系统的默认实现不同：

- Windows `tracert` 通常发送 ICMP Echo；
- Linux `traceroute` 默认可能发送 UDP 高端口；
- 也可使用 `traceroute -I` 或 TCP 模式。

所以不能把“traceroute 一定基于 ICMP”当成普遍规律。

路径中的 `*` 表示该跳没有在超时时间内返回符合预期的响应，可能是路由器过滤、限速、配置不响应或路径变化，并不必然代表网络已经断开。

7.6 ICMP 与路径 MTU

当 IPv4 报文带有 DF 位且超过下一跳 MTU 时，路由器可能返回：

```
ICMP Fragmentation Needed
```

源主机据此降低报文大小或 TCP MSS。若 ICMP 被完全屏蔽，可能出现：

- 小包能通；
- 大包卡住；

- TCP 握手成功但大数据传输停止；
- 浏览器加载部分页面后长时间等待。

这种故障称为 PMTU 黑洞。

Linux 可测试带 DF 的报文：

```
ping -M do -s 1472 -c 1 example.com
```

1472 字节 ICMP 数据加 20 字节 IPv4 首部和 8 字节 ICMP 首部，总长为 1500 字节。

7.7 一次合理的诊断顺序

```
1. dig/getent 验证域名解析
2. ping IP 验证基础可达性
3. ping 域名区分 DNS 与路由问题
4. nc/curl 探测目标端口和应用
5. traceroute 查看路径断点
6. tcpdump/Wireshark 查看 ICMP 与 TCP 行为
7. 检查 NAT、防火墙、MTU 和回程路由
```

例如：

```
dig example.com
ping -c 4 example.com
curl -v --connect-timeout 5 https://example.com/
traceroute example.com
```

如果 `ping` 成功但 `curl` 失败，问题更可能在端口、TLS 或应用层；如果域名解析失败，抓 IP 层报文也不会解决名称问题；如果小包成功、大包失败，则应重点检查 MTU。

8. 本篇小结

```
主机和路由器都查路由表
路由选择以最长前缀匹配为核心
路由结果给出下一跳与接口
目的 IP 不变，TTL 减少
每一跳都要重新封装链路层帧
```

```
ICMP 无端口，用于差错通知和诊断
ping 不能证明具体应用服务可用
traceroute 通过逐级 TTL 探测路径
ICMP 对路径 MTU 发现具有重要作用
```

下一篇进入链路层：IP 数据报怎样被放进以太网帧，交换机怎样根据 MAC 地址把帧送到正确接口。